

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2,BL3)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Case Study

Duration: 2hrs

1. A logistics company is developing an RL-based warehouse robot that learns to pick, pack, and transport items efficiently while avoiding obstacles and minimizing delays. Analyze how the components of a Markov Decision Process (states, actions, rewards, policy, and environment) can be defined for this system. Design a reward function that balances speed, accuracy, and safety, and evaluate how reward design influences the robot's learning and behavior. **(5 Marks)**
2. A streaming platform wants to use Reinforcement Learning to recommend movies dynamically based on user preferences and viewing history. Apply RL concepts to construct a suitable state representation, action space, and reward function. Analyze how different reward signals affect user engagement and evaluate the trade-off between exploration and exploitation. **(5 Marks)**

3. An autonomous agricultural system uses RL to optimize irrigation and fertilizer usage based on soil conditions and weather forecasts. Design an RL framework by defining states, actions, and rewards. Analyze the effect of delayed rewards on crop yield optimization and justify the choice of a suitable learning algorithm. **(5 Marks)**
4. A cybersecurity system uses RL to detect and respond to network attacks in real time. Analyze how states and environment can be modeled for intrusion detection. Design appropriate actions and reward functions, and evaluate the challenges of sparse rewards and real-time decision-making. **(5 Marks)**
5. A ride-sharing company plans to use RL to optimize driver allocation and dynamic pricing strategies. Apply RL concepts to define states, actions, and rewards. Analyze how environmental uncertainty affects learning and evaluate ethical concerns related to pricing strategies. **(5 Marks)**
6. A smart home automation system uses RL to control lighting, heating, and cooling based on occupancy and weather conditions. Design an RL framework including states, actions, and reward function. Analyze how conflicting objectives like comfort and energy saving can be handled and evaluate the impact of reward shaping. **(5 Marks)**
7. A gaming company is developing an RL-based agent to play a complex strategy game against human players. Analyze how the problem can be modeled as an MDP. Design a reward structure that encourages long-term strategy and evaluate how exploration strategies affect performance. **(5 Marks)**
8. A financial institution uses RL to detect fraudulent transactions in real time. Apply RL principles to define the state space, action space, and reward function. Analyze the challenges of imbalanced data and delayed feedback, and evaluate the risks associated with incorrect decisions. **(5 Marks)**
9. A healthcare system uses RL to recommend personalized rehabilitation exercises for patients based on their progress. Design an RL framework with suitable states, actions, and rewards. Analyze the impact of delayed rewards and patient variability, and evaluate ethical concerns and safety considerations. **(5 Marks)**

- 10.A smart grid system uses RL to manage electricity distribution and reduce peak load demand. Analyze how the components of an MDP can be defined in this context. Design a reward function to balance efficiency and reliability, and evaluate the challenges of scalability and real-time learning. **(5 Marks)**
-

Code of Conduct:

*I K.Vijay Bhaskar Reddy(192425155) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

K.Vijay Bhaskar Reddy.

RUBRICS FOR CASESTUDY

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured , and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q1. Warehouse Robot using Reinforcement Learning

Aim

To develop a Reinforcement Learning (RL) based warehouse robot capable of picking, transporting, and delivering items efficiently while avoiding obstacles and minimizing travel time.

Markov Decision Process (MDP)

States

The state consists of the robot's current position, package status (picked/not picked), and obstacle locations within the warehouse.

Actions

The robot can perform four actions:

- Move Up
- Move Down
- Move Left
- Move Right

Environment

The environment is represented as a grid-based warehouse containing pickup points, delivery points, and static obstacles.

Reward Function

- +30 for successfully picking up a package.
- +100 for successfully delivering the package.
- -20 for colliding with an obstacle.
- -1 for each movement to encourage shorter paths.

Policy

The robot learns an optimal navigation policy using the Q-Learning algorithm by maximizing cumulative rewards.

Analysis

The reward function balances efficiency, accuracy, and safety. Positive rewards encourage successful task completion, while penalties discourage collisions and unnecessary movements.

Evaluation

The trained RL agent successfully learns the shortest and safest delivery path, reducing travel time while improving warehouse efficiency.

```

import numpy as np
import random
import matplotlib.pyplot as plt

SIZE=6
PICKUP=(1,1)
DROP=(5,5)
OBSTACLES=[(2,2),(3,2),(4,4)]

actions=["Up","Down","Left","Right"]
moves=[(-1,0),(1,0),(0,-1),(0,1)]

Q=np.zeros((SIZE,SIZE,2,len(actions)))

alpha=0.1
gamma=0.9
epsilon=0.2
episodes=1000
history=[]

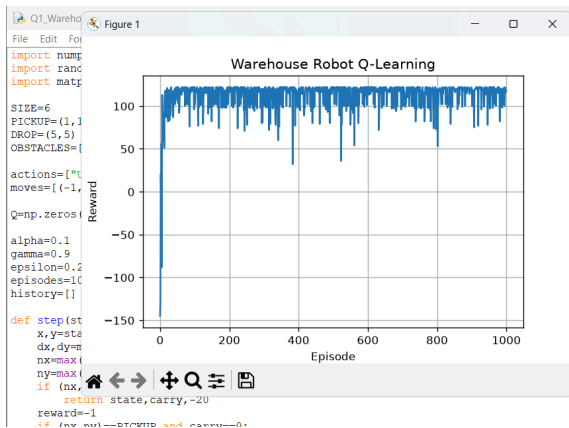
def step(state,carry,action):
    x,y=state
    dx,dy=moves[action]
    nx=max(0,min(SIZE-1,x+dx))
    ny=max(0,min(SIZE-1,y+dy))
    if (nx,ny) in OBSTACLES:
        return state,carry,-20
    reward=-1
    if (nx,ny)==PICKUP and carry==0:
        carry=1
        reward=30
    if (nx,ny)==DROP and carry==1:
        reward=100
        return (nx,ny),carry,reward
    return (nx,ny),carry,reward

for ep in range(episodes):
    state=(0,0)
    carry=0
    total=0
    for _ in range(100):
        x,y=state
        if random.random()<epsilon:
            a=random.randint(0,3)
        else:
            a=np.argmax(Q[x,y,carry])
        ns,ncarry,r=step(state,carry,a)
        nx,ny=ns
        Q[x,y,carry]+=alpha*(r+gamma*np.max(Q[nx,ny,ncarry])-Q[x,y,carry,a])
        state,carry=ns,ncarry
        total+=r
        if state==DROP and carry==1:
            break
    history.append(total)

print("Training Completed")
print("\nLearned Policy")
for i in range(SIZE):
    row=[]
    for j in range(SIZE):
        if (i,j)==PICKUP:
            row.append("P")
        elif (i,j)==DROP:
            row.append("D")
        elif (i,j) in OBSTACLES:
            row.append("X")
        else:
            row.append(actions[np.argmax(Q[i,j,0])][0])
    print(row)

plt.plot(history)
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("Warehouse Robot Q-Learning")
plt.grid()
plt.show()

```



Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
===== RESTART: P:\RL\ASSESSMENT\COL\AT 3\Q1_Warehouse_Robot_Q_Learning.py =====
Training Completed

Learned Policy
[['R', 'D', 'L', 'L', 'U', 'U']
 ['R', 'D', 'L', 'U', 'U', 'U']
 ['U', 'U', 'X', 'U', 'U', 'U']
 ['U', 'L', 'X', 'U', 'U', 'U']
 ['U', 'D', 'R', 'U', 'X', 'U']
 ['D', 'D', 'D', 'U', 'U', 'D']]
```

Q2. Movie Recommendation using Reinforcement Learning

Aim

To develop a Reinforcement Learning based movie recommendation system that dynamically suggests movies according to user preferences and viewing history.

State Representation

The state includes the user's preferred genre, previously watched movies, and historical interactions.

Action Space

The available actions are recommending movies from different genres such as Action, Comedy, Drama, Sci-Fi, and Horror.

Reward Function

- +10 when the user watches or likes the recommended movie.
- -5 when the recommendation is ignored or disliked.

Analysis

Exploration introduces new movie recommendations to discover changing user preferences, while exploitation recommends movies that previously received positive feedback.

Evaluation

The RL agent continuously adapts to user behavior, increasing recommendation accuracy and improving overall user engagement.

```
Q2_Movie_Recommendation_Q_Learning.py - P:\RL\ASSESSMENT\CO1\AT 3\Q2_Movie_Recommendation_Q_Learning.py (3.11.9)
File Edit Format Run Options Window Help

import numpy as np
import random
import matplotlib.pyplot as plt

users=["Action","Comedy","Drama"]
movies=["Action","Comedy","Drama","SciFi","Horror"]

Q=np.zeros((len(users),len(movies)))

alpha=0.1
gamma=0.9
epsilon=0.2
episodes=1000
history=[]

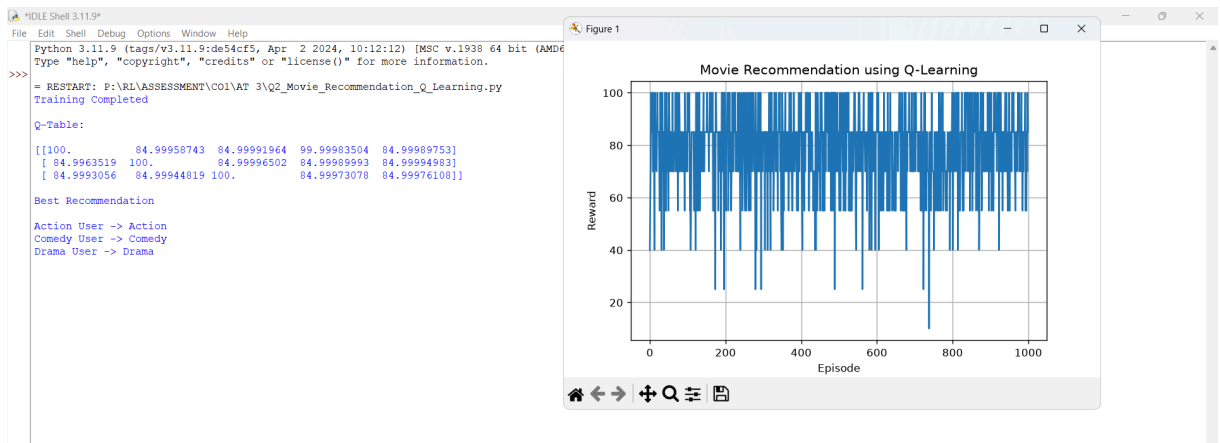
preferences={
0:[0,3],
1:[1],
2:[2]
}

def reward(state,action):
    if action in preferences[state]:
        return 10
    return -5

for ep in range(episodes):
    state=random.randint(0,len(users)-1)
    total=0
    for _ in range(10):
        if random.random()<epsilon:
            action=random.randint(0,len(movies)-1)
        else:
            action=np.argmax(Q[state])
        r=reward(state,action)
        next_state=random.randint(0,len(users)-1)
        Q[state,action]+=alpha*(r+gamma*np.max(Q[next_state])-Q[state,action])
        state=next_state
        total+=r
    history.append(total)

print("Training Completed\n")
print("Q-Table:\n")
print(Q)
print("\nBest Recommendation\n")
for i,u in enumerate(users):
    print(u,"User ->",movies[np.argmax(Q[i])])

plt.plot(history)
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("Movie Recommendation using Q-Learning")
plt.grid()
plt.show()
```



Q3. Smart Agriculture using Reinforcement Learning

Aim

To design a Reinforcement Learning based agricultural system that optimizes irrigation and fertilizer usage based on soil moisture and weather conditions.

States

The environment consists of:

- Soil Moisture (Dry, Normal, Wet)
- Weather Condition (Sunny, Rainy)

Actions

The agent can perform three actions:

- Do Nothing
- Irrigate
- Apply Fertilizer

Reward Function

Positive rewards are assigned for maintaining optimal soil conditions and efficient resource usage, while penalties are applied for unnecessary irrigation or fertilizer application.

Analysis

Crop yield depends on decisions made over multiple time steps; therefore, delayed rewards are an important characteristic of the problem.

Learning Algorithm

Q-Learning is selected because it learns optimal policies without requiring prior knowledge of the environment.

Evaluation

The RL system minimizes water and fertilizer consumption while maximizing long-term crop productivity.

```
*Q3_Smart_Agriculture_Q_Learning.py - P:\RL\ASSESSMENT\CO1\AT 3\Q3_Smart_Agriculture_Q_Learning.py (3.11.9)*
File Edit Format Run Options Window Help

import numpy as np
import random
import matplotlib.pyplot as plt

states=[("Dry","Sunny"), ("Dry","Rainy"), ("Normal","Sunny"), ("Normal","Rainy"), ("Wet","Sunny"),
        ]
actions=["Do Nothing","Irrigate","Add Fertilizer"]

Q=np.zeros((len(states),len(actions)))

alpha=0.1
gamma=0.9
epsilon=0.2
episodes=1000

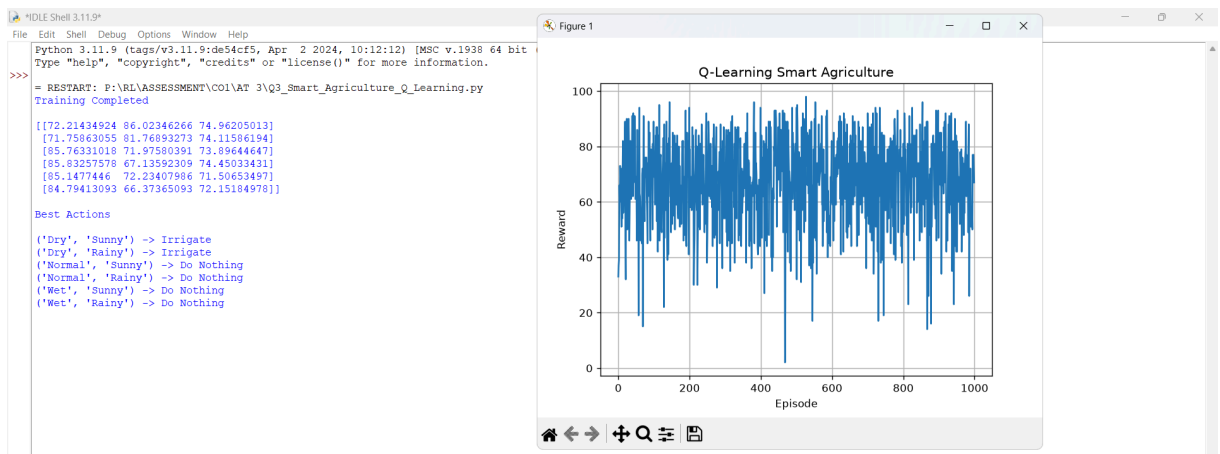
history=[]

def reward_fn(state,action):
    soil,weather=states[state]
    r=0
    if soil=="Dry" and action==1:
        r+=10
    elif soil=="Normal" and action==0:
        r+=10
    elif soil=="Wet" and action==0:
        r+=8
    else:
        r-=5
    if weather=="Rainy" and action==1:
        r-=5
    if action==2 and soil!="Wet":
        r+=3
    return r

for ep in range(episodes):
    s=random.randint(0,len(states)-1)
    total=0
    for _ in range(10):
        if random.random()<epsilon:
            a=random.randint(0,len(actions)-1)
        else:
            a=np.argmax(Q[s])
        r=reward_fn(s,a)
        ns=random.randint(0,len(states)-1)
        Q[s,a]+=alpha*(r+gamma*np.max(Q[ns])-Q[s,a])
        s=ns
        total+=r
    history.append(total)

print("Training Completed\n")
print(Q)
print("\nBest Actions\n")
for i in range(len(states)):
    print(states[i], "->", actions[np.argmax(Q[i])])

plt.plot(history)
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("Q-Learning Smart Agriculture")
plt.grid()
plt.show()
```



Q6. Smart Home Automation using Reinforcement Learning

Aim

To develop a Reinforcement Learning based smart home system that controls lighting, heating, and cooling to maximize comfort while minimizing energy consumption.

States

The state consists of:

- Indoor Temperature (Cold, Comfortable, Hot)
- Occupancy Status (Occupied, Unoccupied)

Actions

The available actions include:

- Turn ON Heater
- Turn ON Air Conditioner
- Turn ON Lights
- Turn OFF Lights
- Do Nothing

Reward Function

Positive rewards are given for maintaining comfortable temperatures and reducing unnecessary electricity consumption, while negative rewards are assigned for inefficient appliance usage.

Analysis

The system must balance two conflicting objectives: occupant comfort and energy efficiency. Reward shaping encourages energy-saving actions without sacrificing user comfort.

Evaluation

The RL agent learns intelligent control policies that improve comfort while reducing electricity usage.

```
Q6_Smart_Home_Q_Learning.py - P:\RL\ASSESSMENT\CO1\AT 3\Q6_Smart_Home_Q_Learning.py (3.11.9)
File Edit Format Run Options Window Help

import numpy as np
import random
import matplotlib.pyplot as plt

states=[(0,0),(0,1),(1,0),(1,1),(2,0),(2,1)]
actions=["Nothing","Heater","AC","Lights OFF","Lights ON"]

Q=np.zeros((len(states),len(actions)))

alpha=0.1
gamma=0.9
epsilon=0.2
episodes=1000

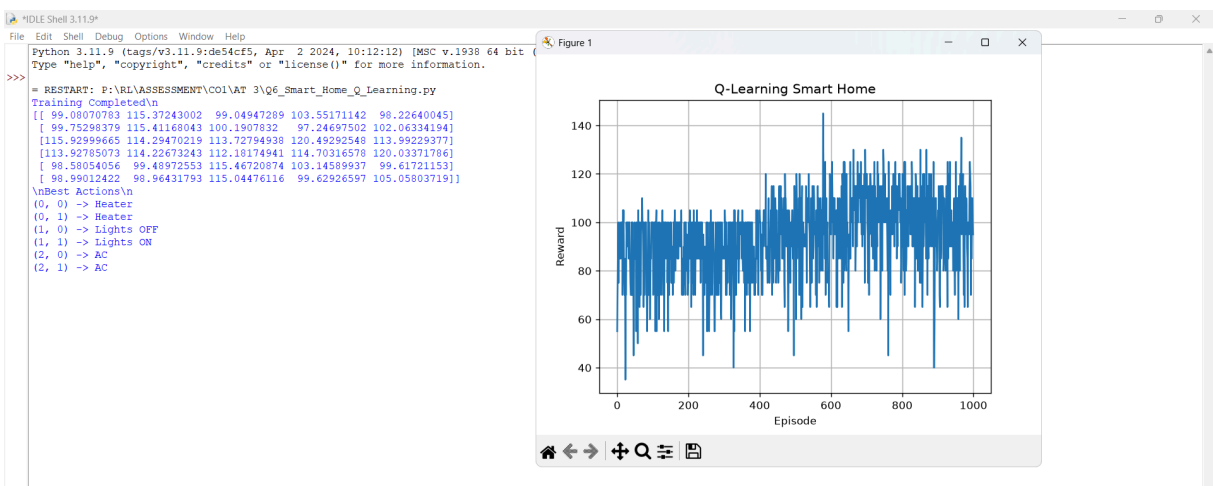
rewards=[]

def get_reward(state,action):
    temp,occ=states[state]
    reward=0
    if temp==1:
        reward+=10
    elif temp==0 and action==1:
        reward+=10
    elif temp==2 and action==2:
        reward+=10
    else:
        reward-=5
    if occ==0 and action==3:
        reward+=5
    if occ==1 and action==4:
        reward+=5
    return reward

for episode in range(episodes):
    state=random.randint(0,len(states)-1)
    total=0
    for step in range(10):
        if random.random()<epsilon:
            action=random.randint(0,len(actions)-1)
        else:
            action=np.argmax(Q[state])
        reward=get_reward(state,action)
        next_state=random.randint(0,len(states)-1)
        Q[state,action]+=alpha*(reward+gamma*np.max(Q[next_state])-Q[state,action])
        state=next_state
        total+=reward
    rewards.append(total)

print("Training Completed\\n")
print(Q)
print("\\nBest Actions\\n")
for i in range(len(states)):
    print(states[i],"->",actions[np.argmax(Q[i])])

plt.plot(rewards)
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("Q-Learning Smart Home")
plt.grid()
plt.show()
```



Q7. Strategy Game Agent using Reinforcement Learning

Aim

To implement a Reinforcement Learning agent capable of learning effective strategies for navigating a grid-based game environment.

Markov Decision Process (MDP)

States

The state includes the player's current position, obstacle locations, and goal position.

Actions

The available actions are:

- Move Up
- Move Down
- Move Left
- Move Right

Environment

The environment is represented as a two-dimensional grid containing obstacles and a target destination.

Reward Function

- +100 for reaching the goal.
- -10 for hitting an obstacle.
- -1 for each movement.

Policy

The Q-Learning algorithm enables the agent to learn the optimal path by maximizing cumulative rewards over repeated episodes.

Analysis

The reward structure encourages long-term planning by rewarding successful goal completion while discouraging inefficient paths and collisions.

Evaluation

The trained agent successfully learns an optimal navigation strategy, demonstrating effective exploration during training and efficient exploitation after convergence.

